Universidade do Algarve

Faculdade de Ciências e Tecnologia

Licenciatura em Engenharia Informática

Sistemas Operativos

Trabalho Prático

Capítulo 3 - Processos

Realizado por: Diogo Carvalho nº 90247

PL: 1

Docente: Daniel José Leonardo Castro

Ano Letivo: 2025/2026

Referencias utilizadas:

Ramey, C., & Fox, B. *Bash Reference Manual*. Edition 5.3, for Bash Version 5.3. Free Software Foundation, May 2025.

MacKenzie, D., et al. *GNU Coreutils Manual*. Version 9.10. Free Software Foundation, 26 Jan. 2026.

Relatório

3.1 Primeira Parte

3.2 Segunda Parte: Exercícios Propostos

1. Para se familiarizar com o ambiente de programação crie um programa que escreva Bom dia a todos! e termine. Compile e execute esse programa. Nota: Em caso de necessidade, altere o código de prog0.c.

Resposta:

```
#include <stdio.h>

int main(){

    printf("Bom dia a todos!\n");

    return(0);

}
```

2. Compile o programa prog1.c e execute-o várias vezes, anotando o output da sua execução. Explique esses resultados e descreva uma forma de os confirmar.

Resposta:

O programa "prog1.c" mostra o identificador do processo atual (PID) e o identificador do processo pai (PPID). Ao executá-lo várias vezes, o valor do "PID" muda, porque cada execução cria um novo processo. Já o "PPID" costuma corresponder ao processo que lançou o programa, normalmente a shell. Uma forma de confirmar isso é comparar o "PPID" apresentado com o identificador da shell que está a executar o programa. O mecanismo do sistema operativo visado é a identificação e hierarquia de processos.

```
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./prog1
Sou um Processo.
Sou o processo 2100595
Fui criado pelo processo 2100555
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./prog1
Sou um Processo.
Sou o processo 2100598
Fui criado pelo processo 2100555
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./prog1
Sou um Processo.
Sou o processo 2100599
Fui criado pelo processo 2100555
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./prog1
Sou um Processo.
Sou o processo 2100600
Fui criado pelo processo 2100555
UALG\a90247@fct-deei-linux:~/SO/3.2$
```

3. Compare o output do programa prog2.c com o anterior e identifique o motivo da diferença.

Resposta:

O "prog2.c" produz um resultado semelhante ao anterior, mas acrescenta também o identificador do utilizador através de "getuid()". A diferença no output deve-se exatamente a essa chamada adicional, que permite saber a que utilizador pertence o processo. O mecanismo visado continua a ser a identificação de processos, agora associada também ao utilizador que os executa.

```
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./prog1
Sou um Processo.
Sou o processo 2100600
Fui criado pelo processo 2100555
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./prog2
Sou um Processo.
Sou o processo 2100607
Fui criado pelo processo 2100555 , a pedido do utilizador 356980996
```

4. Identifique as acções desempenhadas pelo prog3.c e justifique a estrutura que é apresentada.

Resposta:

O "prog3.c" obtém o diretório atual com "getcwd()", mostra esse ponto de partida e depois vai subindo na árvore de diretórios através de "chdir("../")". Em cada passo apresenta o novo diretório e espera por input do utilizador. A estrutura apresentada é hierárquica porque o sistema de ficheiros em Linux está organizado em forma de árvore, partindo da raiz "/". O mecanismo do sistema operativo visado é a gestão de diretórios e navegação no sistema de ficheiros.

```
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./prog3
Ponto de partida: /home/a90247/SO/3.2

         Ponto actual: /home/a90247/SO

         Ponto actual: /home/a90247

         Ponto actual: /home

         Ponto actual: /
```

5. Compile o programa p2.6.c e teste-o com os seguintes exemplos, anotando e comentando os resultados obtidos:

No "p2.6.c", a string de opções é "abs:", o que significa que "-a" e "-b" são opções simples, enquanto "-s" exige um argumento. O programa usa "getopt()" para processar a linha de comandos e, no fim, mostra o valor de "optind" e os argumentos que sobraram. O mecanismo do sistema operativo visado é o processamento de argumentos na execução de programas.

(a) $ p2.6 um dois

Resposta:

Neste caso não existe nenhuma opção válida no início da linha de comandos. Por isso, o programa não encontra opções e apenas deixa os dois argumentos como restantes.

```
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./p2.6 um dois
 optind = 1
Left  um
Left  dois
```

(b) $ p2.6 -a um dois

Resposta:

Aqui a opção "-a" é reconhecida corretamente. Depois disso, os argumentos "um" e "dois" ficam como argumentos restantes.

```
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./p2.6 -a um dois
Found option a
 optind = 2
Left  um
Left  dois
```

(c) $ p2.6 um -a dois

Resposta:

Neste caso, o "getopt()" ainda reconhece a opção "-a", mesmo ela não estando na primeira posição. No fim, ficam como argumentos restantes "um" e "dois".

```
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./p2.6 um -a dois
Found option a
 optind = 2
Left  um
Left  dois
```

(d) $ p2.6 a -a dois b c -s tres -c quatro

Resposta:

A opção "-a" é reconhecida. A opção "-s" também é reconhecida, mas como o argumento dado é "tres", a conversão com "atoi()" produz "0". A opção "-c" não existe na string de opções, por isso é assinalada como inválida. Os restantes argumentos ficam no fim.

```
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./p2.6 a -a dois b c -s tres -c quatro
Found option a
Found option s
       with an argument of 0
Found an option that was not in optstring
 optind = 5
Left  a
Left  dois
Left  b
Left  c
Left  quatro
```

(e) $ p2.6 a -a dois b c -s 10 tres -c quatro

Resposta:

O comportamento é semelhante ao caso anterior, mas agora o argumento de "-s" é "10", pelo que "atoi()" devolve esse valor. A opção "-c" continua a ser inválida.

```
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./p2.6 a -a dois b c -s 10 tres -c quatro
Found option a
Found option s
       with an argument of 10
Found an option that was not in optstring
 optind = 5
Left  a
Left  dois
Left  b
Left  c
Left  tres
Left  quatro
```

6. Compile e execute o programa prog4.c e explique os resultados obtidos com o programa. Retire o comentário à linha 23, compile e execute novamente o programa.

Resposta:

O "prog4.c" imprime algumas mensagens e depois executa "fork()". A partir desse momento passam a existir dois processos: o processo pai e o processo filho. Como ambos continuam a execução a partir da instrução seguinte ao "fork()", a mensagem "a todos" é impressa duas vezes. Isto acontece porque tanto o pai como o filho executam essa linha. Ao retirar o comentário da última linha, passa também a ser mostrado o identificador de cada um dos dois processos, permitindo distinguir melhor pai e filho. O mecanismo do sistema operativo visado é a criação de processos com "fork()".

```
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./prog4
Sou um Processo.
Sou o processo 2100743, filho de 2100555
Bom dia ...
a todos
a todos
UALG\a90247@fct-deei-linux:~/SO/3.2$ gcc -Wall -o prog4 prog4.c
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./prog4
Sou um Processo.
Sou o processo 2100767, filho de 2100555
Bom dia ...
a todos
Sou o processo 2100767, filho de 2100555
a todos
Sou o processo 2100768, filho de 2100767
```

7. Explique os resultados obtidos com o programa p2.1.c.

Resposta:

O "p2.1.c" mostra informação sobre o processo inicial e, dentro de um ciclo, cria novos processos com "fork()". Sempre que um filho é criado, este imprime o seu "PID", o "PPID" e o identificador do grupo de processos. O pai usa "wait()" para esperar que o filho termine antes de continuar, o que evita que todos os processos avancem de forma totalmente desordenada. O resultado demonstra a relação entre pai e filho e mostra também que os processos pertencem ao mesmo grupo. O mecanismo do sistema operativo visado é a criação e sincronização de processos, bem como a noção de grupo de processos.

```
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./p2.1


 Initial process         PID 2100802     PPID 2100555    GID 2100802

New process 0            PID 2100803     PPID 2100802    GID 2100802
New process 1            PID 2100804     PPID 2100803    GID 2100802
New process 2            PID 2100805     PPID 2100804    GID 2100802
New process 2            PID 2100806     PPID 2100803    GID 2100802
New process 1            PID 2100807     PPID 2100802    GID 2100802
New process 2            PID 2100808     PPID 2100807    GID 2100802
New process 2            PID 2100809     PPID 2100802    GID 2100802
```

8. Sabendo que a chamada ao sistema fork devolve 0 (zero) no processo filho e o número do novo processo criado no processo pai, crie o programa prog5.c a partir de prog4.c de forma sejam emitida a mensagem Processo Pai ou Processo filho de forma consistente. Formate a mensagem de forma a incluir o identificador de cada processo.

Resposta:

```
/* Sistemas Operativos 2025/26
 * Data: 17/Mar/2026
 * Autor: Diogo Carvalho
 * Objectivo: Distinguir processo pai de processo filho após fork()
 * Compilar com: gcc -Wall -o prog5 prog5.c
 * Executar com: ./prog5
 */

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main(void)
{
   pid_t pid;

   printf("Sou um Processo.\n");
   printf("Antes do fork: processo %d, filho de %d\n", getpid(), getppid());

   pid = fork();

   if (pid < 0) {
```

```
        perror("fork");
        return 1;
    }
    else if (pid == 0) {
        printf("Processo Filho: PID = %d, PPID = %d\n", getpid(), getppid());
    }
    else {
        printf("Processo Pai: PID = %d, criou o filho com PID = %d\n", getpid(), pid);
    }

    return 0;
}
```

```
UALG\a90247@fct-deei-linux:~/SO/3.2$ ./prog5
Sou um Processo.
Antes do fork: processo 2101010, filho de 2100987
Processo Pai: PID = 2101010, criou o filho com PID = 2101011
Processo Filho: PID = 2101011, PPID = 2101010
```

3.3 Terceira Parte: Questões gerais e exercícios adicionais~

3.3.1 Questões teóricas

As questões que a seguir se apresentam são de índole teórica e visam relembrar alguns dos tópicos já apresentados.

1. Quais são e em que consistem as duas funções principais de um

sistema operativo?

Resposta:

As duas funções principais de um sistema operativo são a gestão dos recursos do sistema e a disponibilização de um ambiente de execução para os programas. Isto significa que o sistema operativo controla recursos como o processador, a memória e os dispositivos, ao mesmo tempo que facilita a interação entre o utilizador, as aplicações e o hardware.

2. O que é uma mudança de contexto e em que circunstâncias pode ocorrer?

Resposta:

A mudança de contexto ocorre quando o sistema operativo interrompe a execução de um processo e passa a executar outro. Para isso, guarda o estado do processo atual e restaura o estado do processo seguinte. Este mecanismo pode acontecer, por exemplo, quando um processo termina, fica bloqueado ou perde o processador para outro.

3. Descreva sucintamente os seguintes conceitos no âmbito de sistemas operativos:

(a) processos

Resposta:

Um processo é um programa em execução. Para além do código, inclui também os dados, os registos e os recursos necessários ao seu funcionamento.

(b) bloqueios

Resposta:

Os bloqueios acontecem quando um processo não pode continuar a executar porque está à espera de um recurso, de uma operação de entrada/saída ou de outro evento.

(c) gestão de memória

Resposta:

A gestão de memória consiste no controlo da atribuição e utilização da memória pelos processos. O sistema operativo garante que cada processo usa o seu espaço de forma organizada e protegida.

(d) Entradas e saídas

Resposta:

As entradas e saídas correspondem à comunicação entre o sistema e os dispositivos externos, como o teclado, o monitor, o disco ou a impressora.

(e) Ficheiros

Resposta:

Os ficheiros são estruturas usadas para guardar informação de forma persistente. O sistema operativo permite criá-los, organizá-los, lê-los e modificá-los.

(f) Segurança

Resposta:

A segurança tem como objetivo proteger o sistema e os dados, através de mecanismos como autenticação, permissões e controlo de acessos.

(g) shell

Resposta:

A shell é o interpretador de comandos que permite ao utilizador comunicar com o sistema operativo e executar comandos ou scripts.

4. Hierarquize pela velocidade de acesso os dispositivos:

(a) disco rígido

(b) processador

(c) impressora

(d) RAM

(e) memória cache

Resposta:

A ordem correta, da maior para a menor velocidade de acesso, é: processador, memória cache, RAM, disco rígido e impressora. Esta ordem reflete a proximidade e a rapidez com que cada componente responde ao sistema.

5. Justifique a diversidade de sistemas operativos do ponto de vista do equipamento empregue e do ponto de vista do utilizador de cada tipo de sistema.

Resposta:

A diversidade de sistemas operativos explica-se pelas diferenças entre os equipamentos e também pelas necessidades dos utilizadores. Dispositivos diferentes, como computadores, telemóveis ou servidores, exigem sistemas adaptados, tal como diferentes utilizadores valorizam características distintas, como simplicidade, desempenho ou segurança.

6. Um processo pode estar em diferentes estados. Descreva os estados de processos e a transição entre esses estados.

Resposta:

Um processo pode passar por vários estados: novo, pronto, em execução, bloqueado e terminado. Estas mudanças acontecem conforme o processo é criado, recebe o processador, espera por um recurso ou conclui a sua execução.